(1) Explain different Symbol Table organisations

Symbol table is an important data structure created and maintained by the compiler in order to keep track of semantics of variables.

There are different ways Symbol tables are organised or implemented:

(i) List

→ In this method, an array is used to store name and associated information.

→ A pointer available is maintained at end of all stored and new names are added in the order as they arrive.

→ To search for a name we start from the beginning of the list till available pointer and if not found, we get an error "use of undeclared name".

→ While inserting a new name we must ensure that it is not allready present otherwise an error "multiple defined name

→ Insertion is of order O(1) but lookup is of order O(n)

→ takes only minimum amount of space.

(ii) Linked List

→ This implementation uses a linked list. Link field is added to each of record.

→ Searching of names is done in order pointed by link of the link field.

→ A pointer "First" is maintained to point to the first record of the symbol table.

→ Insertion is of order O(1) but lookup of order O(n)

(iii) Hash table

→ In hashing scheme, two tables are maintained - hash table and symbol table.

→ Hash table - array of index : 0 to (table size - 1)
These entries are pointers pointing to names of symbol table.

→ Hash functions are used to search for a name.

→ Insertion and lookup can be made fast O(1)
→ Advantage is, quick to search
Disadvantage is, complicated to implement.

(iv) Binary Search Tree.

→ Another approach to implement a symbol table is to use a binary search tree. We add two link fields (left & right child)

→ All names are created as child of the root node that always follows the property of binary search tree.

→ Insertion and lookup of order $O(\log_2 n)$

2) Explain about different storage allocation strategies

Various storage allocation strategies are:

(i) Static storage allocation

→ In static allocation, names are bound to storage locations.

→ If memory is created at compile time then memory will be created in static area and only once.

→ Static allocation supports dynamic data structure that means memory is created only at compile time and deallocated after program completion.

→ The drawback with static storage allocation is that the size and position of data objects should be known at compile time.
Another drawback is restriction of recursion procedure.

(ii) Stack Storage Allocation

→ Static storage organised as stack

→ An activation record is pushed into stack when activation begins and it is popped when activation ends.

→ Activation record contain locals so that they are bound to fresh storage in each activation record. The value of locals is deleted when activation ends.

→ It works on the basis of LIFO

→ And this allocation supports recursion process.

(iii) Heap Storage Allocation

→ This is the most flexible allocation scheme

→ Allocation and deallocation of memory can be done at any time and place depending on user's requirement

→ Heap allocation is used to allocate memory to the variable dynamically and when the variables are no more used then claim it back.

→ Heap storage allocation supports recursion process.

(3) What is meant by scope rules of a language. Describe how access to non-local names can be made in block structured language. Write an example.

Scoping rules of a programming language dictate how symbol names which are found in the body of a scope (function, macro, class --) but not defined there, are resolved.

Several different strategies are:

→ Disallowing them together

→ Only two scoping levels.

→ Lexical scoping

→ Dynamic scoping.

A non-local variable can be accessed by Block Structured Languages by, Static scope or lexical scope. In this type of scope, the scope is verified by examining the text of the program.

- Block - New scope with a sequence of statements that contain the local data declaration. It is enclosed within the delimiter.
- Nesting depth - defines how many function blocks it is nested into

(i) Access link - pointers used in the implementation of lexical scope which is obtained by using points to each activation record.

If procedure p is nested within a procedure q then access link of p points to access link of most recent activation record of procedure q.

So non-local data reference for any procedure can be found out by descending chain of pointers to find all statically enclosing procedures.

- example of access link:

```
main()
{ int a=100

    fun2(int b)
    { int c = a+b
      return c
    }

    fun1(int b)
    { return fun2(2*b) }

    fun1(a);  ---------> answer = 300
}
```

Nested procedure include Activation Record access link that enables user to access the AR of most recent action taken by immediate outer procedure.

here access link of fun1 and fun2 points to main.

The follows normal static scope rules

(ii) Displays:

→ If access links are used in search, then search can be slow. So optimize it to give location without a search.

→ Display is a global array d of pointers to activation records, indexed by lexical nesting depth.

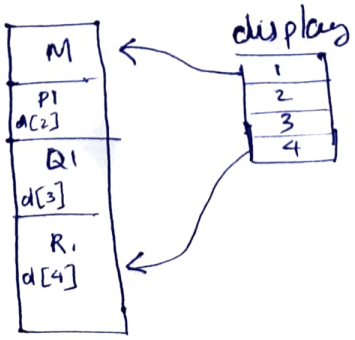
d[i] is an array element which points to most recent activation of block at nesting depth.

→ A non-local X can be found in following manner:

- Use one array access to find the activation record containing X. If the most-closely nested declaration of X is at nesting depth 1, the d[i] points to activation record containing location for X.

→ Use relative addressing within activation record.

- Example for display



Display can be maintained:

- In Regesters
- Statically allocated memory
- Store display and create new copy on each entry.